SkillGraph

Team Project Execution and Responsibility Report

Project title:	SkillGraph: An Explainable Skill Relationship Graph for University-to-Industry Skill Gap Analysis and Personalized Learning Path Recommendation
Team size:	2 members
Duration:	10 weeks
Project type:	Data science research and full-stack application development
Research areas:	Data science, NLP, knowledge graphs, recommendation systems, software engineering

Contents

1 Project Overview **3**

2 Research Objectives **3**

3 Project Scope **3**

4 Team Organization **4**

5 Milestone 1: Research and Planning (Weeks 1–2) **4**

6 Milestone 2: Data Collection and Preparation (Weeks 3–4) **5**

7 Milestone 3: Development and Implementation (Weeks 5–8) **5**

7.1 Sprint 1: NLP Skill Extraction (Week 5) 5

7.2 Sprint 2: Skill Normalization and Knowledge Graph (Week 6) 6

7.3 Sprint 3: Student Skill Gap Analysis (Week 7) 6

7.4 Sprint 4: Recommendation and Product Integration (Week 8) 6

7.5 Learning topics and completion criteria 7

8 Milestone 4: Evaluation and Finalization (Weeks 9–10) **7**

9 Data Science Learning Roadmap **7**

10 Weekly Collaboration Schedule **8**

11 GitHub Collaboration Workflow **8**

12 Technology Stack **9**

13 Final Product Requirements **9**

14 Research Evaluation Framework **10**

15 Definition of Done **10**

16 Final Team Agreement **10**

1 Project Overview

Many university students have a vague idea of what employers want but no clear picture of how their own skills compare. SkillGraph is our attempt to close that gap. It is a career and skill platform that reads real job descriptions, pulls out the technical skills they ask for, and organizes those skills into a knowledge graph. A student can then enter what they already know, see what is missing for a target role, and get a suggested order for learning the rest, along with reasons for each suggestion.

The project has two goals that run side by side. The first is a research goal: we want to find out how well NLP and knowledge graph techniques can support explainable skill gap analysis and learning path recommendation. The second is a product goal: we want to build a working web application that students can actually use to explore career requirements, find their gaps, and decide what to learn next.

Both of us will take part in the data science learning, the experiments, the implementation, the evaluation, and the writing. We are not splitting the project into two separate halves.

2 Research Objectives

Over the ten weeks we plan to collect and analyze real job descriptions, build an NLP pipeline that extracts technical skills from them, and map the many ways people write the same skill onto standard skill concepts. From there we will build a knowledge graph linking skills and occupations, design a way to assess a student's current skills and measure the gap, and build a recommendation system that produces personalized and explainable learning paths. Finally, we will evaluate all of this through experiments and user testing.

We organized the work around four research questions.

- **RQ1:** How well can NLP techniques extract and normalize technical skills from unstructured job descriptions?
- **RQ2:** How can a knowledge graph represent the relationships between occupations, skills, and competencies?
- **RQ3:** How much does graph-based analysis improve skill gap identification compared with ordinary keyword matching?
- **RQ4:** How well can graph-based recommendations produce relevant learning pathways that students can understand?

3 Project Scope

To keep the project manageable, we start with five data-related occupations: Data Analyst, Business Intelligence Analyst, Data Scientist, Data Engineer, and Machine Learning Engineer. These roles overlap enough to make the skill comparisons interesting, but they are still close enough to each other that one taxonomy can cover them.

Our target is roughly 300 to 500 English-language job descriptions, though the real number will depend on what we can legally collect. For each posting we want to record the job title, the company or source, the full description, the required and preferred skills, the experience requirements, the location, the collection date, and a reference to where it came from.

We will only use data sources we are allowed to access. For every source we will write down where the data came from and check its terms and licensing before collecting anything, and again before we share or redistribute it.

4 Team Organization

We work with a mix of shared learning and individual ownership. Each of us leads certain parts of the project, but neither of us is allowed to treat the other's part as a black box.

Member 01 (Data Science and NLP Lead) owns the data and language side of the project. That means collecting and cleaning the job descriptions, running the exploratory data analysis, building skill extraction and normalization, developing the skill taxonomy, constructing the knowledge graph, calculating skill importance, running the gap analysis, and evaluating the NLP experiments.

Member 02 (Recommendation and Product Engineering Lead) owns the annotation work and the application. That covers annotating and validating skill labels, building the database, integrating the graph data, writing the recommendation algorithm, building the student skill profile, the FastAPI backend, the Next.js frontend, and the D3.js graph visualization, and finally testing and deploying the whole product.

On top of this, we share a set of responsibilities. We study data science concepts together, take part in the EDA and statistical analysis, and review each other's extraction and normalization results. We also validate the graph relationships together, make sure both of us understand the recommendation algorithm, review each other's code, reproduce each other's experiments, discuss our findings, and write the paper and prepare the presentation as a pair.

We want to be clear that "primary ownership" only decides who starts a task and who is first to be asked about it. It does not mean the other person can ignore it. By the end, both of us should understand the whole pipeline.

5 Milestone 1: Research and Planning (Weeks 1–2)

The goal of the first two weeks is to lay the foundation before we write any real code: the research direction, the scope, the data requirements, and the system architecture.

Member 01 will review existing work on skill extraction and job market analysis, look for possible job description sources, and study the ESCO and O*NET skill taxonomies. Member 01 will also look into NLP techniques for extracting technical skills and use that to define what our skill extraction needs to do. The expected outputs are a literature review, a data source report, an initial skill taxonomy, and notes on NLP methods.

Member 02 will look at existing career and skill recommendation platforms, design the system architecture and the PostgreSQL schema, sketch the user interface and workflow, and research ways to visualize graphs. The outputs here are a competitor analysis, a system architecture diagram, a database ER diagram, and initial UI wireframes.

Together we will define the research problem and finalize the research questions, decide how we will evaluate the work and which baselines we will compare against, define student proficiency levels, and agree on the scope and tech stack. This should give us an approved project proposal, a final set of research questions, an evaluation strategy, and a proficiency assessment framework.

This is also where we start learning the basics together: research methodology, how to do a

literature review, how to write a good research question, the Python data science workflow, and a first look at NLP and knowledge graphs.

We will consider Milestone 1 finished when the proposal is complete, the research questions are final, the literature review has been started and documented, the data sources have been identified and checked, the architecture and database schema are designed, and both of us can explain the scope of the project.

6 Milestone 2: Data Collection and Preparation (Weeks 3–4)

In these two weeks we build the dataset and get to know it. A model is only as good as its data, so we want to understand the structure and the quality problems before moving on.

Member 01 will collect the job descriptions (aiming for 300 to 500 if feasible), organize the raw records, remove duplicates, clean the text, and look at missing values. After that, Member 01 will run the exploratory data analysis and look at how often skills appear for each occupation. The outputs are the raw and cleaned datasets, the preprocessing scripts, an EDA notebook, a data quality report, and skill frequency charts.

Member 02 will write the annotation guidelines, annotate an initial sample of job descriptions, and review and validate the skill labels. Member 02 will also build the first version of the skill dictionary, set up the PostgreSQL tables, and design how data gets imported. The outputs are the annotation guidelines, a labeled dataset, the skill dictionary, an annotation quality report, and the database itself.

Some tasks we do together. Both of us will independently annotate the same sample of 30 job descriptions, then compare where we disagreed and measure how much we agreed. We will also review the skill categories, look at how job requirements differ between occupations, check the overall quality of the dataset, and sign off on it for the NLP experiments. This gives us an annotation comparison, an agreement analysis, an occupation comparison report, and an approved research dataset.

The learning topics for these weeks are Python, Pandas and NumPy, data cleaning, missing-value analysis, descriptive statistics, frequency distributions, data visualization, EDA, and data annotation with quality control.

Milestone 2 is done when the dataset is collected with its sources recorded, duplicates are removed, missing values are investigated, the EDA is finished, the annotation guidelines and skill taxonomy are final, the database is ready, and both of us can explain the EDA results.

7 Milestone 3: Development and Implementation (Weeks 5–8)

This is the longest milestone. We implement the research methods and connect them into a working SkillGraph application. We split it into four one-week sprints, and in each one both of us work on the same theme from different angles.

7.1 Sprint 1: NLP Skill Extraction (Week 5)

Member 01 builds the text preprocessing, implements a dictionary-based skill extractor, and experiments with spaCy's PhraseMatcher to pull technical skills out of the job descriptions, then evaluates the results. Member 02 builds utilities for evaluating extraction against the annotated

data, goes through the extracted skills to find false positives and false negatives, starts the skill management API, and builds a simple interface for reviewing the extraction output.

Together we compare the different extraction approaches, look into the cases where the extractor got it wrong, discuss precision, recall, and F1-score, and decide which method is worth improving. The deliverables are the extraction pipeline, a baseline, an evaluation dataset, and an NLP performance report.

7.2 Sprint 2: Skill Normalization and Knowledge Graph (Week 6)

Member 01 develops skill normalization, so that different ways of writing the same skill map to one canonical concept. Member 01 also tries semantic similarity, generates relationships between skills, and builds the first knowledge graph with NetworkX. Member 02 writes the graph data access layer and the graph API endpoints, builds a prototype of the interactive graph visualization showing how occupations and skills connect, and reviews the quality of the graph data.

As a pair, we validate the normalization, review the relationship types, and work out how to tell prerequisites apart from skills that are merely related or that just tend to appear together in the same posting. We also check that the graph is consistent. The deliverables are a canonical skill dictionary, a normalized dataset, the knowledge graph, the visualization prototype, and a relationship validation report.

7.3 Sprint 3: Student Skill Gap Analysis (Week 7)

Member 01 calculates how important each skill is for each occupation, looks at skill frequency and relevance, implements the gap calculation, compares requirements across the target occupations, and studies how the gaps are distributed. Member 02 builds the student skill profiles and the proficiency input, connects the profiles to the backend, builds the gap dashboard, and shows which skills a student already has and which are missing.

Together we review how skill importance is calculated, validate the proficiency levels, compare our graph-based analysis against plain keyword matching, and discuss the assumptions and limitations. The deliverables are the importance scores, the gap analysis engine, the student profile module, the dashboard, and the comparison experiment.

7.4 Sprint 4: Recommendation and Product Integration (Week 8)

Member 01 looks at how skills should be prioritized when career opportunities are taken into account, identifies the important dependencies in the graph, validates the priority calculations, and reviews the recommended learning sequences and their explanations. Member 02 implements the recommendation engine and learning path generation, builds the learning path interface, finishes the Next.js dashboard, and integrates FastAPI with D3.js.

Together we check whether the recommendations are relevant, whether the learning sequences make sense, and whether the explanations are clear. We also test the whole application and fix whatever breaks during integration. At the end of this sprint we should have a recommendation engine, explainable recommendations, an interactive SkillGraph, an integrated backend and frontend, and a working prototype.

7.5 Learning topics and completion criteria

Across the four sprints we are learning NLP fundamentals, text preprocessing, skill extraction, semantic similarity, feature engineering, graph theory, NetworkX, recommendation systems, how to evaluate algorithms, API integration, and software testing.

We will count Milestone 3 as complete when extraction, normalization, the knowledge graph, the student profile, the gap analysis, the recommendation engine, and the interactive graph are all implemented, the frontend and backend work together, and both of us understand the full pipeline.

8 Milestone 4: Evaluation and Finalization (Weeks 9–10)

The last two weeks are for checking how well everything works, validating our findings, finishing the product, and preparing the final presentation.

Member 01 handles the data science evaluation. That means evaluating the NLP extraction, comparing the dictionary-based method with the improved one, calculating precision, recall, and F1-score, evaluating the skill importance calculation, analyzing the gap results, and preparing the experiment tables and figures. The outputs are an NLP evaluation report, a baseline comparison, a statistical analysis, and the experiment results.

Member 02 handles the recommendation and product evaluation. That means evaluating how relevant the recommendations are, checking that the prerequisites are consistent, judging the quality of the learning paths, running usability tests, testing the backend and frontend, and preparing deployment. The outputs are a recommendation evaluation report, a user testing report, an application test report, and the deployed application.

Together we review all the experimental results, write the methodology, implementation, and experiments sections of the paper, discuss findings and limitations, write the abstract and conclusion, prepare the presentation, and do a last reproducibility check. The final outputs are the research paper, the presentation, a product demonstration, the documentation, and experiments that can be reproduced.

The learning topics here are precision, recall, and F1-score, statistical evaluation, experimental design, baseline comparison, interpreting results, research writing, and technical presentation.

Milestone 4 is complete when the NLP, skill gap, and recommendation methods have all been evaluated, user testing is done where feasible and approved, the results and limitations are documented, the application is tested and deployed, and the paper and presentation are finished.

9 Data Science Learning Roadmap

We both follow the same learning roadmap, so that neither of us falls behind on the theory while the other is busy with implementation. Each week's topic is tied to something we are actually doing in the project.

Week	Topic	Practical application
1	Research methodology	Research questions
2	Python and Pandas fundamentals	Data inspection
3	Data cleaning	Job data preprocessing
4	EDA and statistics	Skill distribution analysis
5	NLP fundamentals	Skill extraction
6	Text similarity and normalization	Skill matching
7	Graph theory	Knowledge graph
8	Recommendation systems	Learning path
9	Evaluation and statistics	Experiment comparison
10	Research communication	Paper and presentation

For each topic we will write down a learning objective, do a practical exercise, and explain how it connects to SkillGraph. We will keep a GitHub notebook or a documented implementation, and each of us will give a short explanation of the topic to the other.

10 Weekly Collaboration Schedule

We use the same weekly routine throughout the project.

Day	Activity	Who
Monday	Study concepts and plan tasks	Both
Tuesday	Implement assigned tasks	Individual
Wednesday	Continue implementation	Individual
Thursday	Exchange code and review	Both
Friday	Run experiments and tests	Both
Saturday	Discuss findings and improve work	Both
Sunday	Update GitHub and plan next week	Both

As a rough guide, each of us should spend about 1.5 hours a week on data science learning, 3 to 4 hours on implementation, and 1 hour on code review. On top of that, about 30 minutes goes to explaining what we learned and 15 minutes to updating GitHub. These numbers can change around exams and coursework.

11 GitHub Collaboration Workflow

All work is tracked through GitHub Issues and Pull Requests. For each task we start from an issue, create a feature branch, do the work, write tests, and document the results. Then we open a pull request and assign the other person as reviewer, deal with their comments, and merge into `main` only after approval. We do not commit directly to `main` for normal development work.

Branch names follow the pattern `feature/<topic>`. Examples are `feature/data-collection`, `feature/eda-analysis`, `feature/nlp-extraction`, `feature/skill-normalization`, `feature/knowledge-gr` `feature/skill-gap-analysis`, `feature/recommendation-engine`, `feature/backend-api`, `feature/student` and `feature/interactive-graph`.

Commit messages start with a short prefix that says what kind of change it is: `feat:` for new functionality, `fix:` for bug fixes, `data:` for dataset changes, `research:` for experiments, `test:` for tests, `docs:` for documentation, and `refactor:` for code improvements.

12 Technology Stack

Component	Technology
Programming language	Python
Data processing	Pandas, NumPy
Data visualization	Matplotlib, Plotly
Statistics	SciPy, Statsmodels
NLP	spaCy
Semantic similarity	Sentence Transformers
Machine learning	Scikit-learn
Graph processing	NetworkX
Database	PostgreSQL
Backend	FastAPI
Frontend	Next.js, TypeScript
UI styling	Tailwind CSS
Graph visualization	D3.js
Research environment	Jupyter Notebook
Version control	Git and GitHub
Deployment	Docker

The first version of the system will not depend on any paid AI APIs. We will use local NLP and graph-processing methods so that anyone can reproduce our results.

13 Final Product Requirements

The finished application should have seven main parts:

- **Career Dashboard**, an overview of the student's career readiness.
- **My Skills**, where students see their proficiency and the evidence behind it.
- **Career Explorer**, for looking at job requirements and comparing occupations.
- **SkillGraph**, the interactive visualization of how skills relate to each other.
- **Gap Analysis**, showing which competencies a student has and which are missing.
- **Learning Paths**, with prioritized learning recommendations.
- **Research Insights**, showing aggregate skill trends across the industry.

Whenever the system recommends a skill, it must be able to explain why. It should say how important the skill is for the target occupation, which prerequisite knowledge is needed first, and which career opportunities the skill may open up.

14 Research Evaluation Framework

Each part of the research is evaluated in a way that fits what it produces.

Component	Evaluation method
Skill extraction	Precision, recall, F1-score
Skill normalization	Accuracy against reviewed labels
Knowledge graph	Relationship validity and coverage
Skill gap analysis	Comparison with human-reviewed profiles
Recommendation	Relevance, ordering, explanation quality
Application	Task completion and usability feedback

We keep the evaluation data separate from the development data, and we make sure that similar or duplicate job descriptions do not end up on both sides, since that would inflate our results. We also need to be honest about student proficiency: it is self-reported, and we will label it that way instead of presenting it as objectively verified.

15 Definition of Done

We only mark a task as finished when all of the following are true.

- ☐ The implementation works.
- ☐ The code is organized and understandable.
- ☐ The tests pass.
- ☐ The documentation is updated.
- ☐ The results can be reproduced, where that applies.
- ☐ The other member has reviewed it.
- ☐ The pull request is approved and merged.

Data science tasks have a few extra requirements:

- ☐ The data sources are documented.
- ☐ The methods and assumptions are explained.
- ☐ The evaluation results are recorded.
- ☐ The limitations are discussed.
- ☐ Both of us can reproduce and explain the work.

16 Final Team Agreement

We both agree to take part in every milestone, to learn data science together, and to finish our assigned tasks on the agreed timeline. We will share what we learn and review each other's work, keep our GitHub documentation clear, avoid merging major changes that nobody has reviewed, and make sure our results can be reproduced. Both of us will also contribute to the final paper and presentation.

If everything goes as planned, after ten weeks we will have a documented job description dataset, an NLP pipeline for skill extraction and normalization, a validated skill knowledge graph, a method for analyzing student skill gaps, and an explainable learning path recommendation engine. We will also have a complete interactive web application, our experimental results, a research paper, and a final presentation with a live demo.

Finishing SkillGraph is not the only point. Each of us should also be able to explain the data science methods on our own, understand how the system is built, reproduce the experiments, and defend the research findings.